

Foundations of Digital Design

CHAPTER 5



15th April 2026

KJSSE

1

Synchronous Sequential Circuit

- A general block diagram of a clocked sequential circuit is shown in Fig. 7.1.
- This is also known as a finite state machine (FSM).
- Depending upon the way the external outputs are obtained from the circuit, there are two different models of sequential circuits.
- These are Mealy model and Moore model.

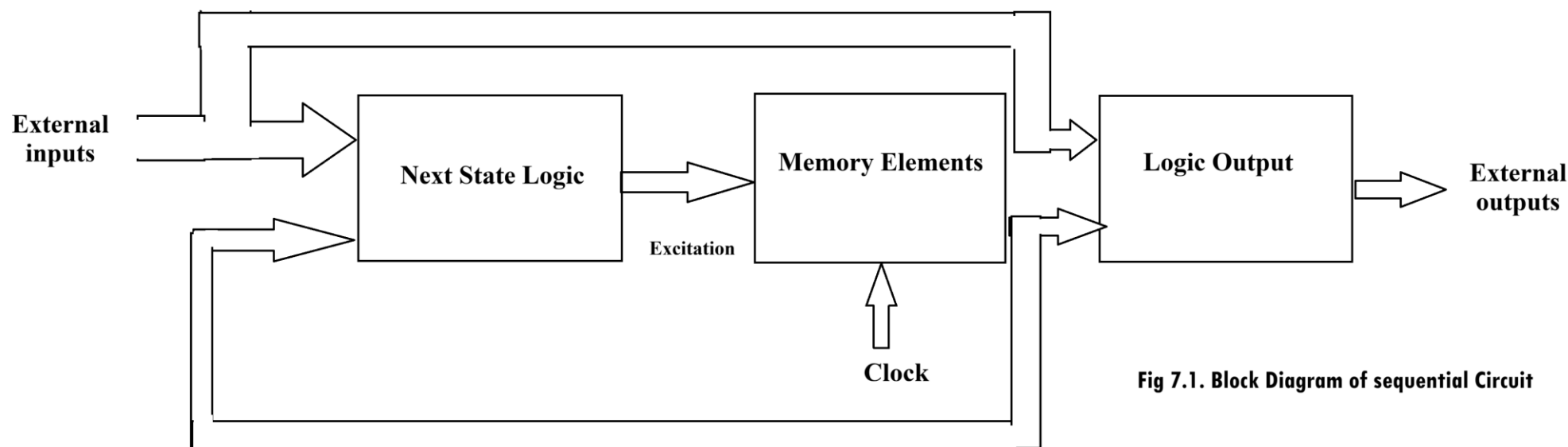
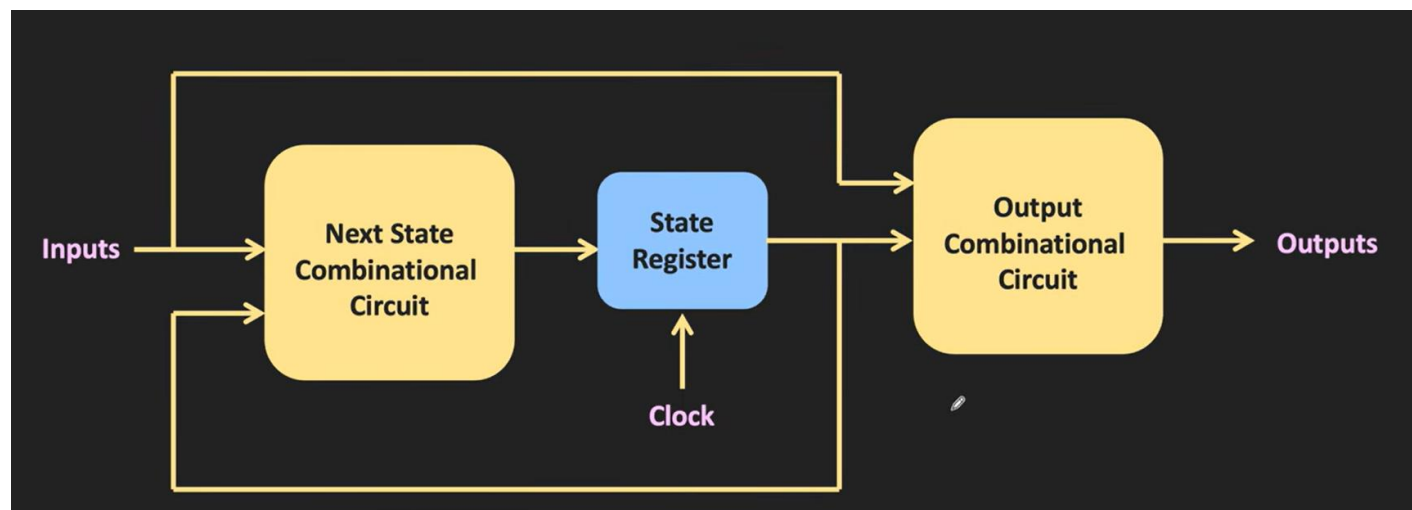


Fig 7.1. Block Diagram of sequential Circuit

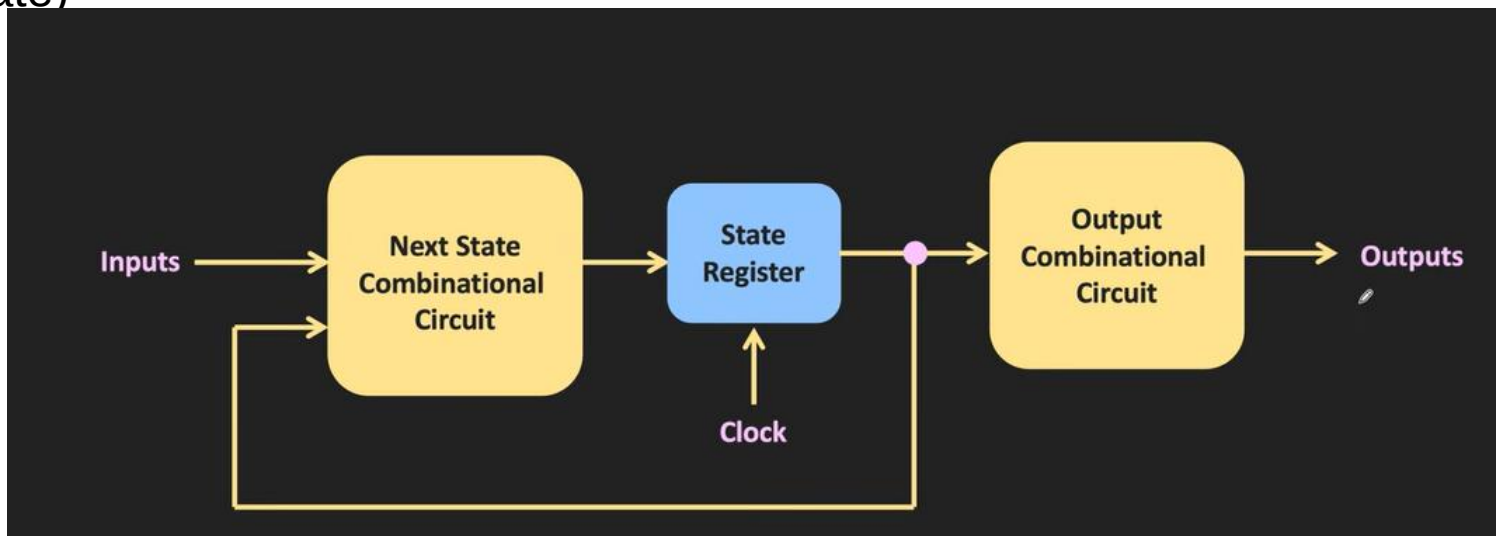
Mealy Model

- In the case of a Mealy model, the next state is a function of the present state and the present inputs.
- Its output is also a function of the present state and the present inputs.
- Figure BELOW represents Mealy model.
- In general, the next state and the output of a Mealy model are uniquely defined by:
- Next State = F_1 (Present state, inputs)
- Outputs = F_2 (Present state, inputs)



MOORE MODEL

- The block diagram of a Moore model of circuit is shown in Fig. 8.38. Similar to the Mealy model, the next state in a Moore model is a function of the present state and inputs, but the outputs are functions only of **the present state** and are independent of the inputs.
- Therefore, the Moore model is defined as:
- **Next state** = $F1$ (Present state, inputs)
- **Outputs** = $F2$ (Present state)



COMPARISON BETWEEN MOORE AND MELAY MODEL

Moore Machine	Mealy Machine
Output depends only upon the present state.	Output depends on the present state as well as present input.
Moore machine also places its output in the state.	Mealy Machine places its output on the transition.
More states are required.	Less number of states are required.
Moore machines requires more hardware requirement for circuit implementation.	Mealy Machines requires less hardware requirement for circuit implementation.
They react slower to inputs(One clock cycle later).	They react faster to inputs.
Synchronous output and state generation.	Asynchronous output generation.
Output is placed on states.	Output is placed on transitions.
Easy to design.	It is difficult to design.
Has more or the same states as that of the Mealy machine.	Has fewer or the same states as that of the Moore machine.

FINITE STATE MACHINE

- When referring to the two models, some books or technical resources refer to a sequential circuit as a FINITE STATE MACHINE or 'FSM'
- Therefore, the two models are referred to as Moore FSM or Mealy FSM
- The word FINITE is used to indicate that there are limited number of states, which can be attained by the Finite State Machine

Advantages of FINITE STATE MACHINE

- FSMs are simple to implement
- After development and realization, the testing of FSM is also easy
- The time required to design, implement, and test an FSM is short
- FSM is a well-proven modelling technique for the synchronous sequential circuit
- FSMs are flexible meaning we can implement them in many different ways

Disadvantages of FSM

- FSM is not suitable for solving all types of problems. It can be used only if we can decompose the system behaviour into distinct states. The transition among those states must be well defined and known to the designer
- FSMs are predictable, which makes them useless in some applications, like computer gaming
- FSMs become too complicated for larger systems and become difficult to manage

State Machine Notations

- Input variable: All the variables generated outside the sequential circuit are called input variables
 - Output variable: All the variables that leave the sequential circuit are called output variables
 - State variable: As the output of flipflops (memory) defines the state of a sequential machine, we can define the state variables as the outputs of flipflops
- ∴ State variables = Outputs of flip-flops
- Excitation variable: Excitation variables are defined as the inputs to the memory (flip-flops). This name is given because the variable excites the memory to change its state. State variables are produced by the excitation variables
- ∴ Excitation variables = Inputs to flip-flops

State Machine Notations

- State: The state of a sequential machine is dependent on the content of memory. If the memory is realized using flipflops, then the machine state is defined by the Q output of flipflops.

State = Q output of flip-flops

The relation between state variables and the number of states is:

$$y = 2^x$$

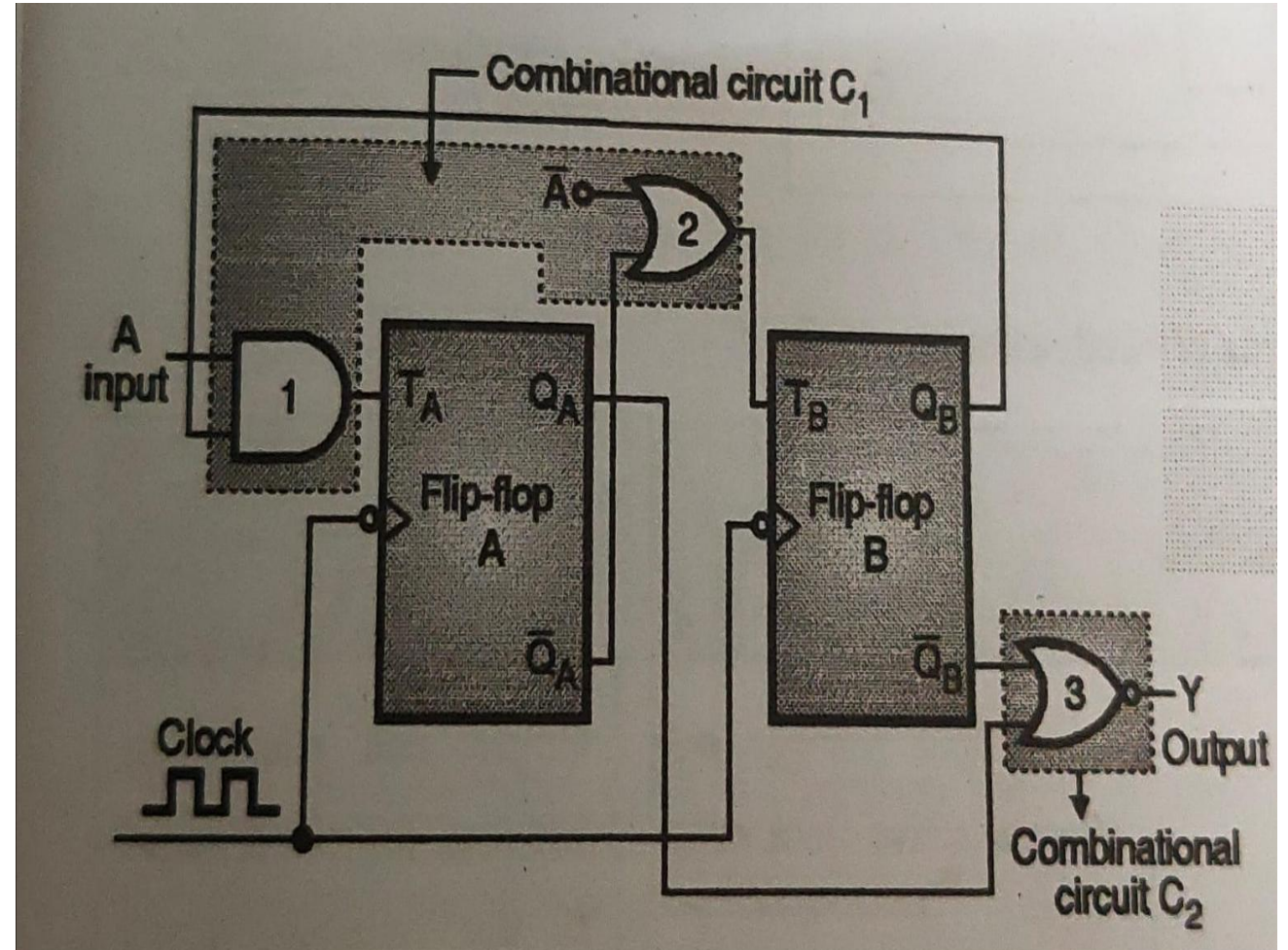
Where, y = Maximum number of possible states and

x = Number of state variables

Question

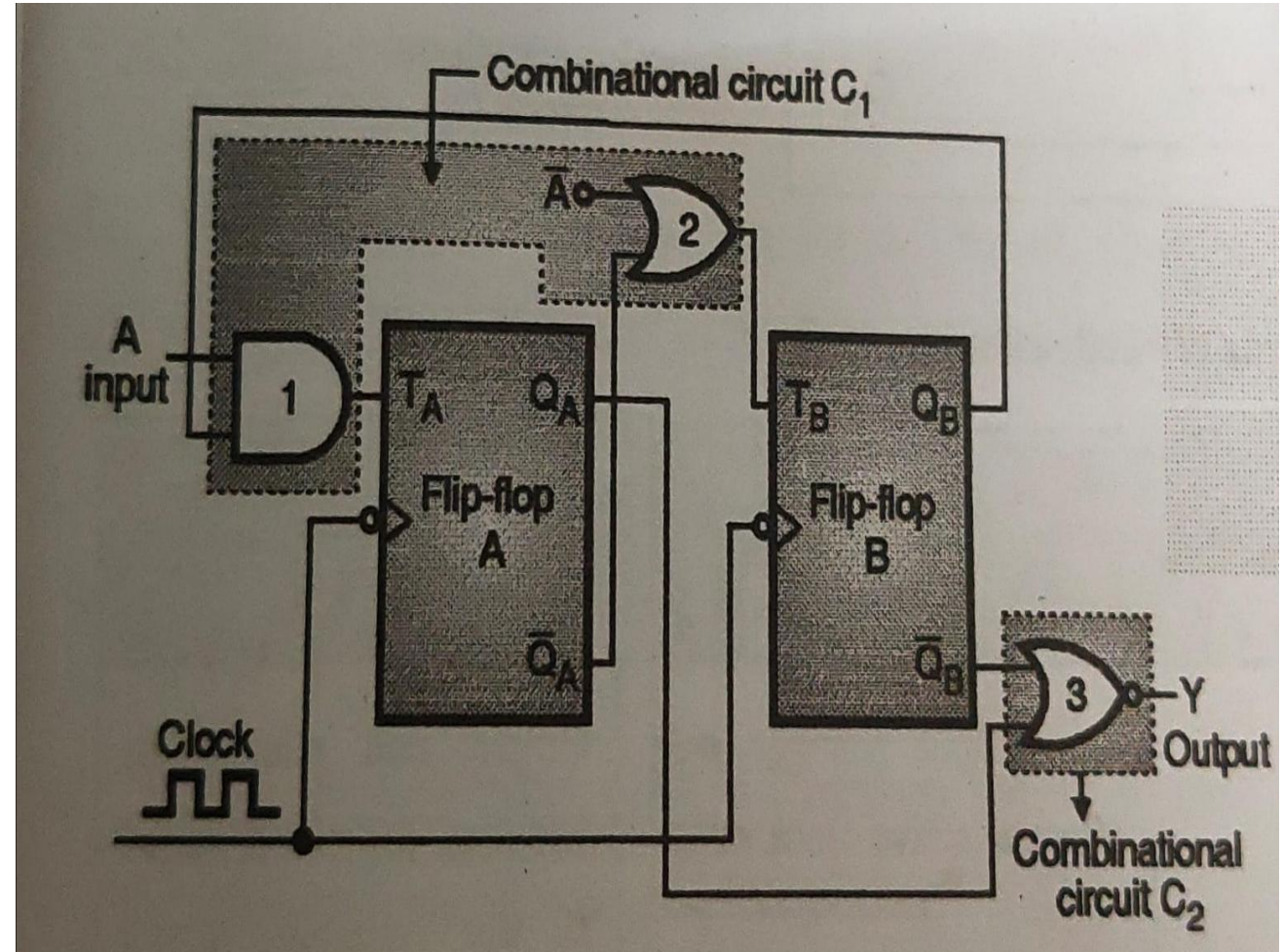
List the following for the given question

1. Input variable
2. Output variable
3. State variable
4. Excitation variable
5. Maximum number of possible states

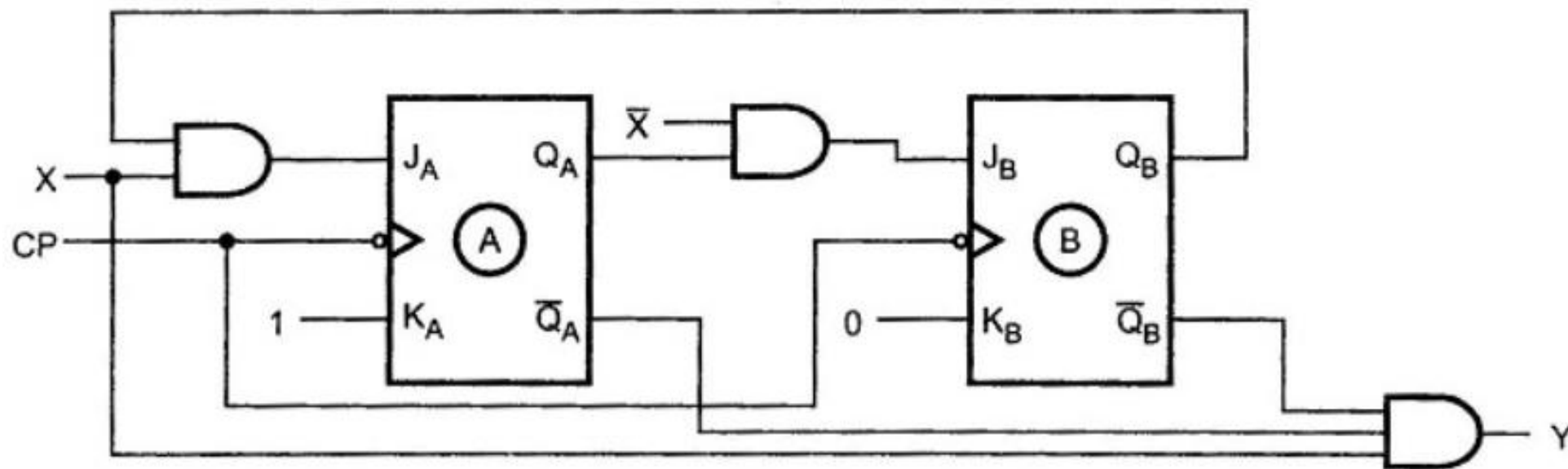


Answer

Category	Variable(s)
Input	A
Output	Y
State	Q_A, Q_B
Excitation	T_A, T_B
Max States	4



Question: Identify the type of circuit



Present states, Next states

- Present state: The status of all the state variables at any instant say "t" before we apply the next clock edge is called a present state.
- Next state: The status of all state variables at the time instant say (t+1), just after we have applied the active clock edge is called as next state.

State table

- We use a truth table to explain the relation between input and output for a combinational circuit
- But it is not suitable to describe the input-output relation of a sequential circuit
- Instead, we use the state table to describe the input-output relation of the sequential circuits.
- The basic building block of a sequential circuit is a flip-flop.

State table

- The outputs Q_A , Q_B etc. of such flip flops will be used as state inputs to a sequential circuit.
- They are also called as state variables.
- In addition to this "x" represents an external input and Y represents the output of the sequential circuit.
- Y will be dependent on the state variables (Q_A , Q_B ) and the external input x.
- The general format of a state table is shown in Table 1.5.1.

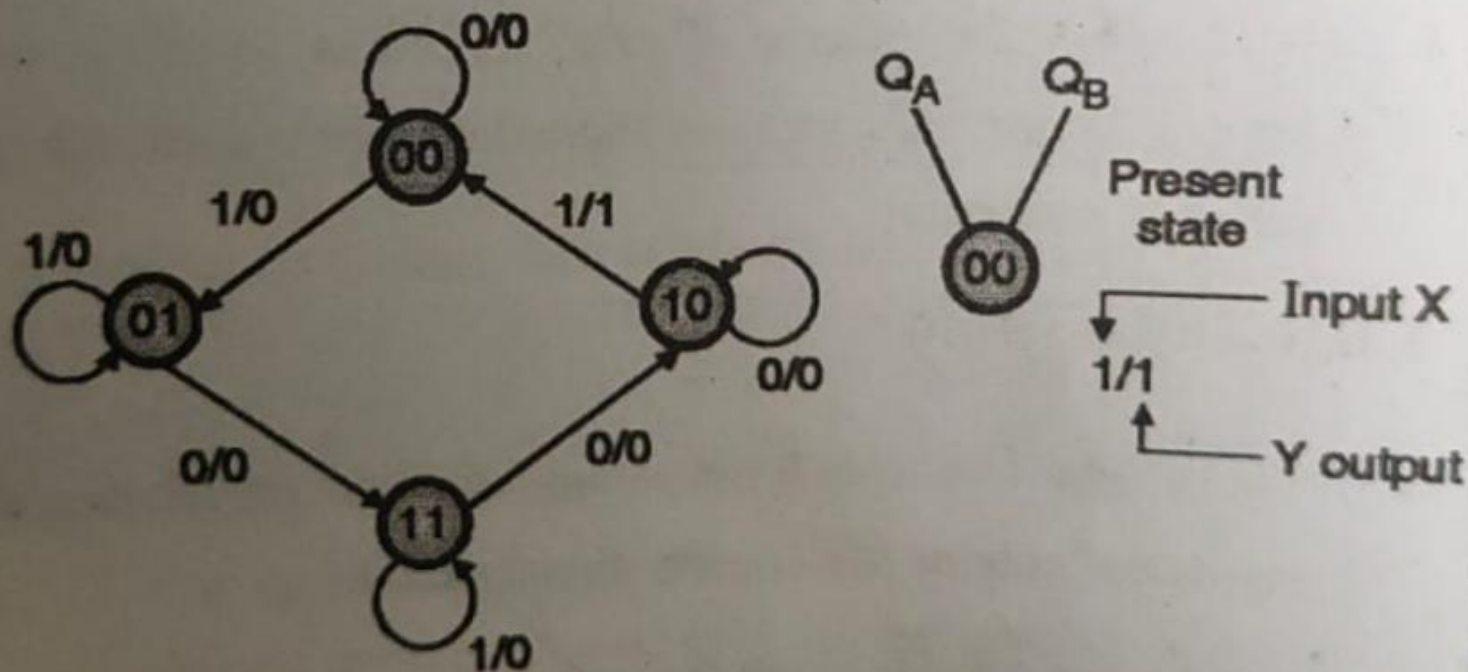
Table 1.5.1 : General format of state table

Present state	Next state		Output Y	
	x=0	x=1	x=0	x=1
Q_A Q_B	Q_A Q_B	Q_A Q_B		
0 0	0 0	0 1	0	0
0 1	1 1	0 1	0	0
1 0	1 0	0 0	0	1
1 1	1 0	1 1	1	0

State Diagram

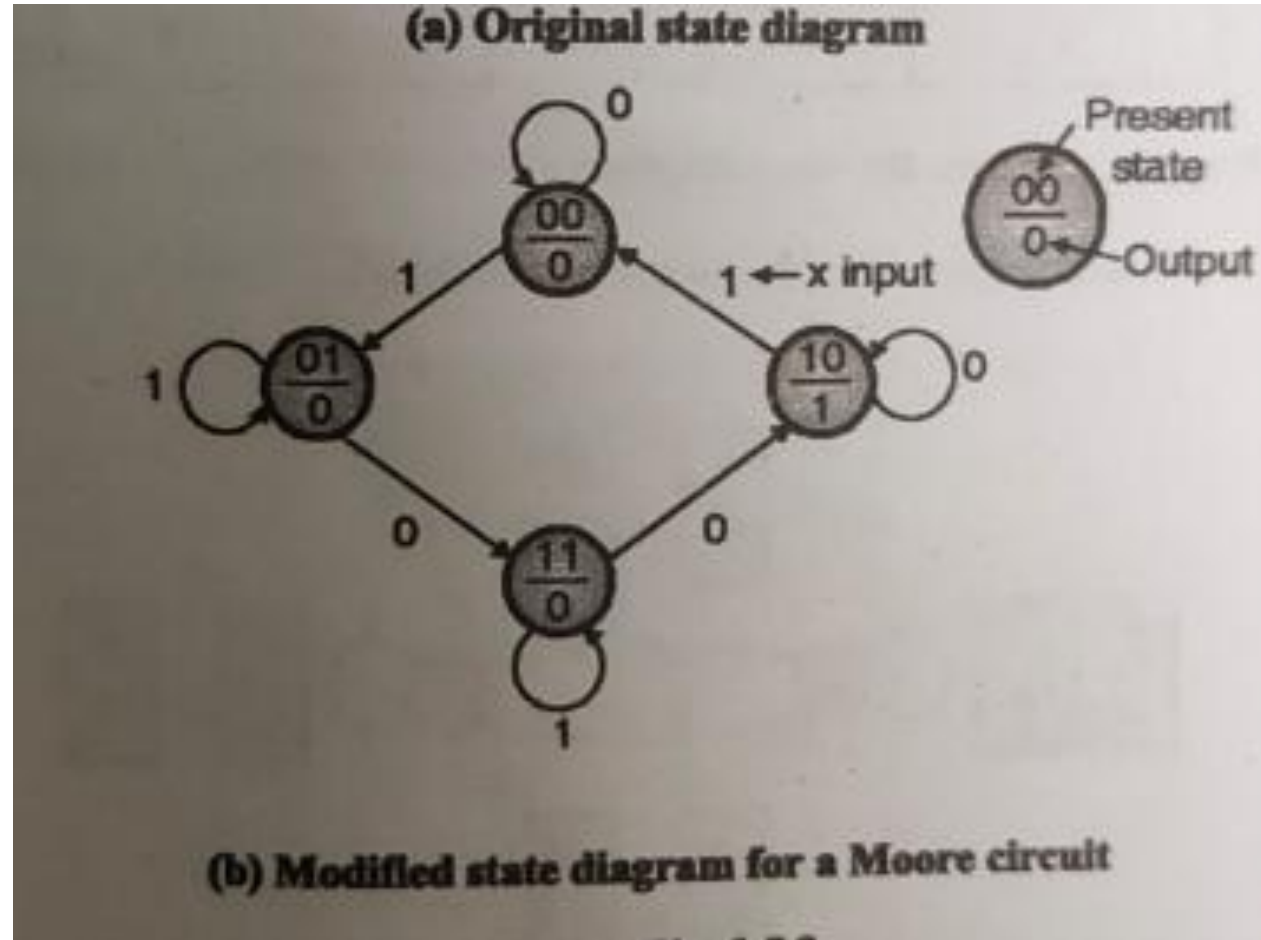
- The state transition and the behaviour of a finite state machine can be represented in a graphical form that is known as the state diagram of the finite state machine.
- They visualise how a system moves between, at most, a finite number of conditions based on external inputs
- **States:** Represented by **circles or rounded rectangles**, indicating the current condition or "state" of the system
- **Transitions:** **Solid arrows connecting states**, representing the change from one state to another triggered by an event or input

State Diagram –Melay model



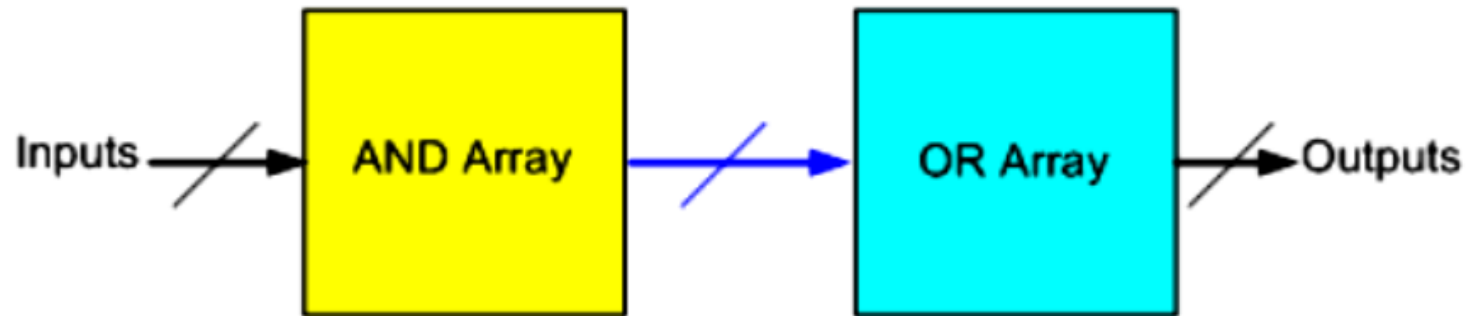
(C-653) Fig. 1.5.1 : State diagram

State Diagram – Moore model



Programmable Logic Devices

- An IC that contains large numbers of gates, flip-flops, etc. that can be configured by the user to perform different functions is called a Programmable Logic Device (PLD).
- The internal logic gates and/or connections of PLDs can be changed/configured by a programming process.
- Programming the device involves blowing the fuses along the paths that must be removed in order to obtain the particular configuration of the desired logic function.
- PLDs are typically built with an array of AND gates (AND-array) and an array of OR gates (OR-array)



Programmable Logic Devices

Problems of using standard ICs:

- Problems of using standard ICs in logic design are that they require hundreds or thousands of these ICs, considerable amount of circuit board space, a great deal of time and cost in inserting, soldering, and testing. Also require keeping a significant inventory of ICs.

Advantages of using PLDs:

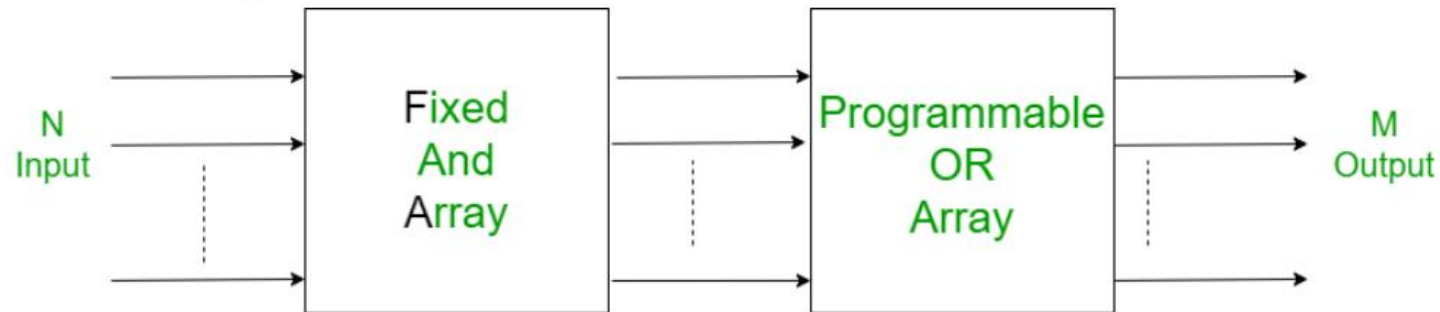
- Advantages of using PLDs are less board space, faster, lower power requirements (i.e., smaller power supplies), less costly assembly processes, higher reliability (fewer ICs and circuit connections means easier troubleshooting), and availability of design software.

There are three fundamental types of standard PLDs: PROM, PAL, and PLA.

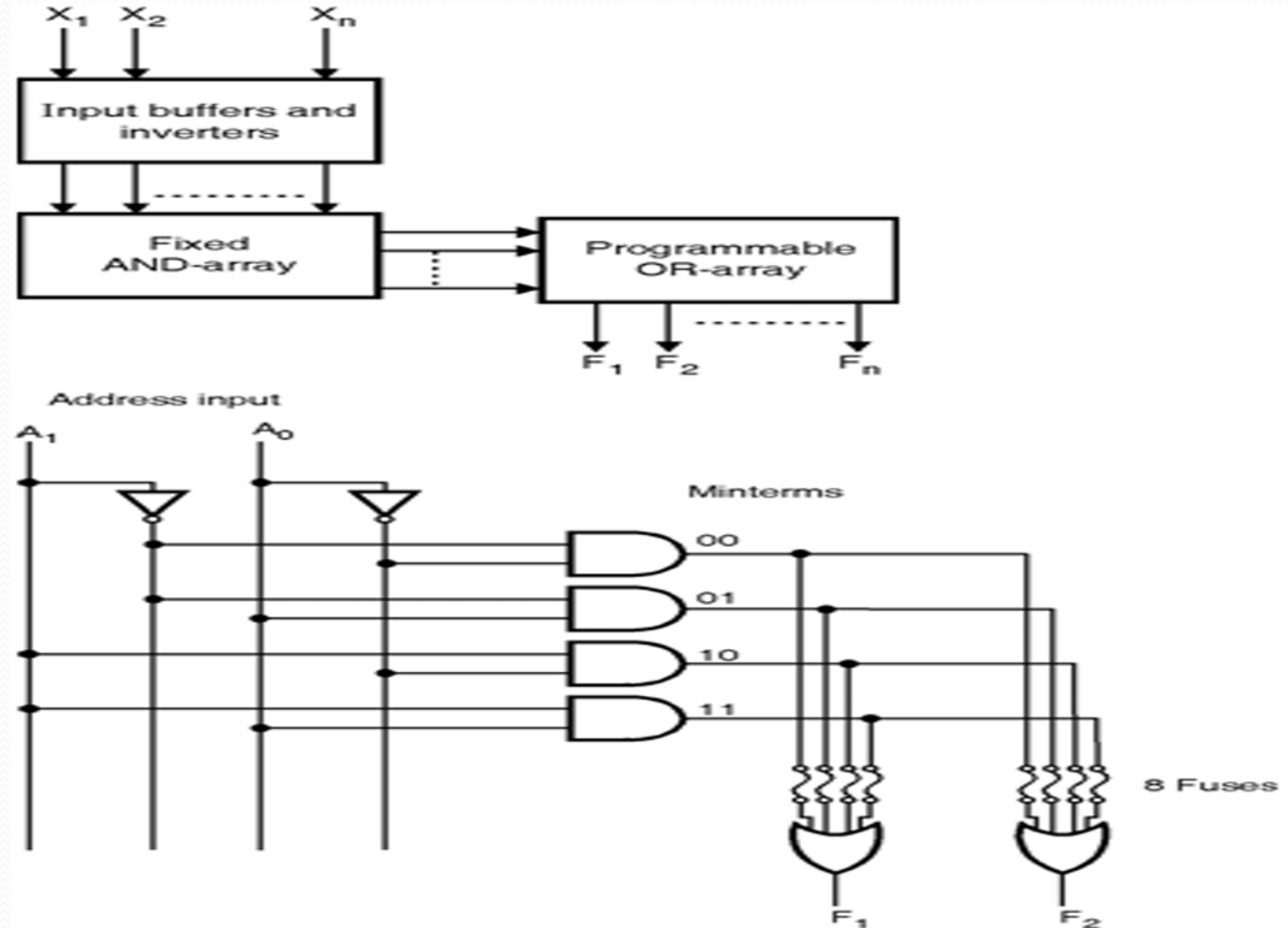
PROM

- PROM stands for Programmable Read-Only Memory. It is a memory chip on which information can be composed as it were once.
- Once a program has been composed onto a PROM, it remains there until the end of time. Not at all like Ram, PROMs hold their substance when the computer is turned off.

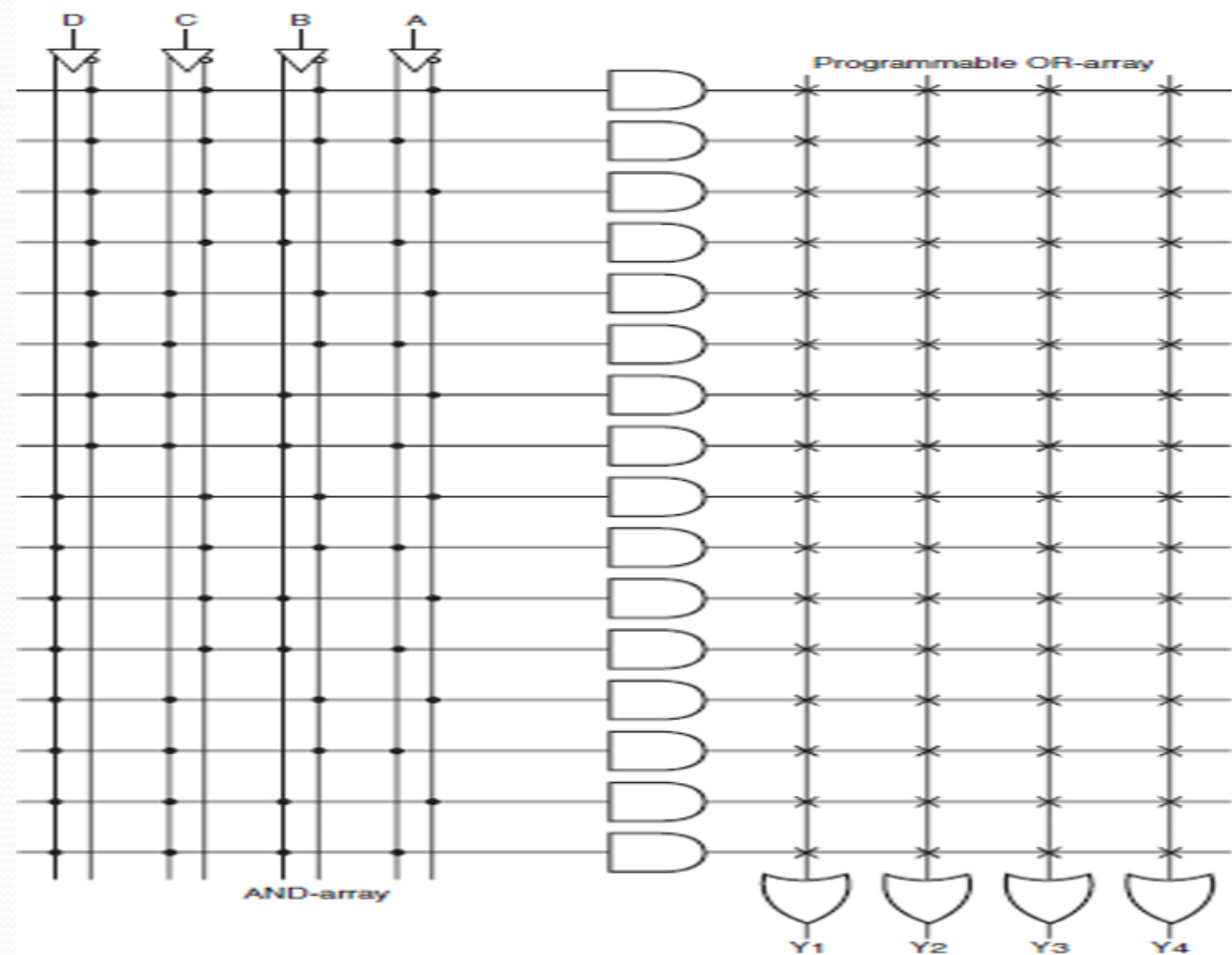
Block Diagram of PROM



PROM



Internal architecture of a PROM having four input lines, a hard-wired array of 16 AND gates and a programmable array of four OR gates.

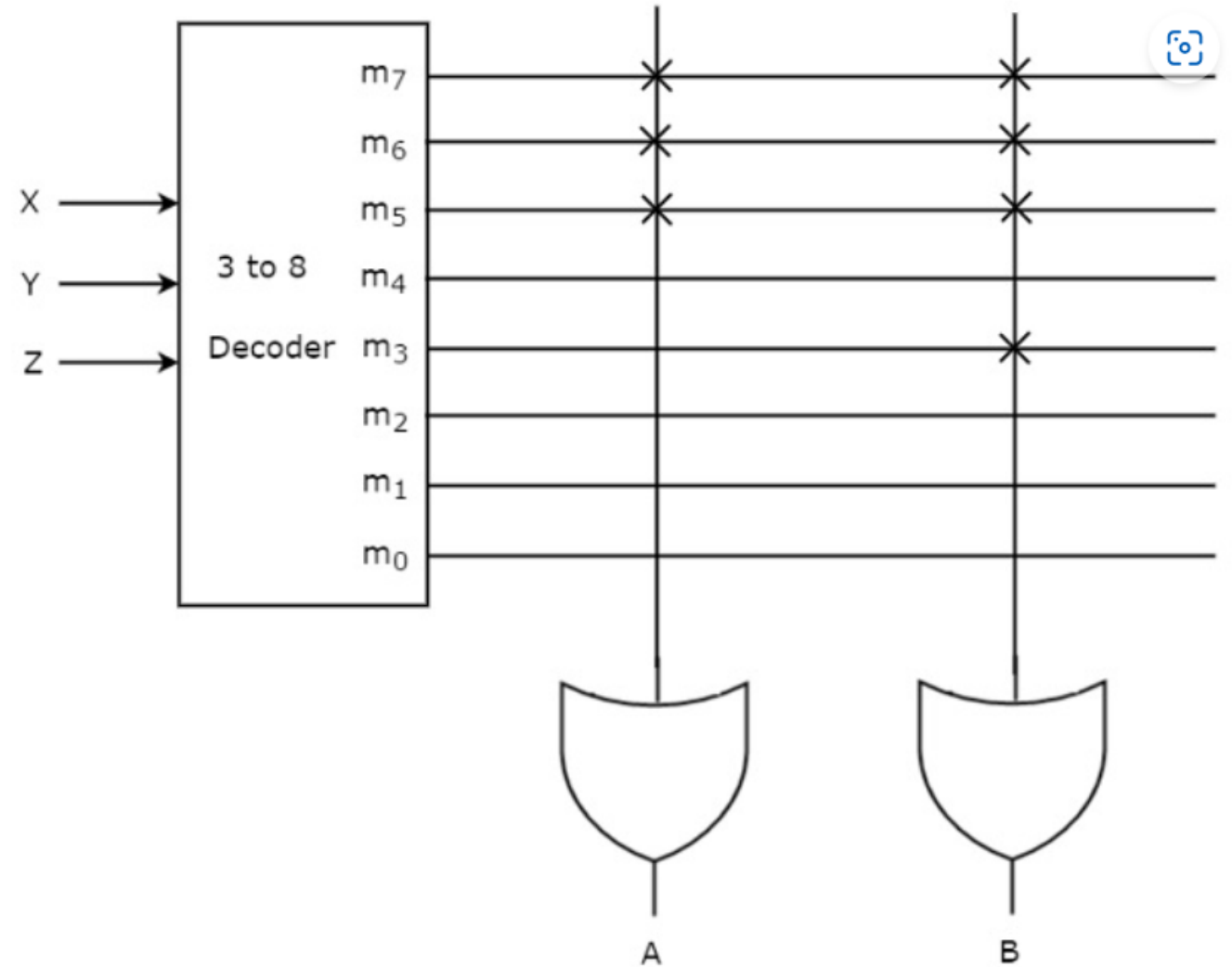


Numerical

Implement the following Boolean functions using PROM.

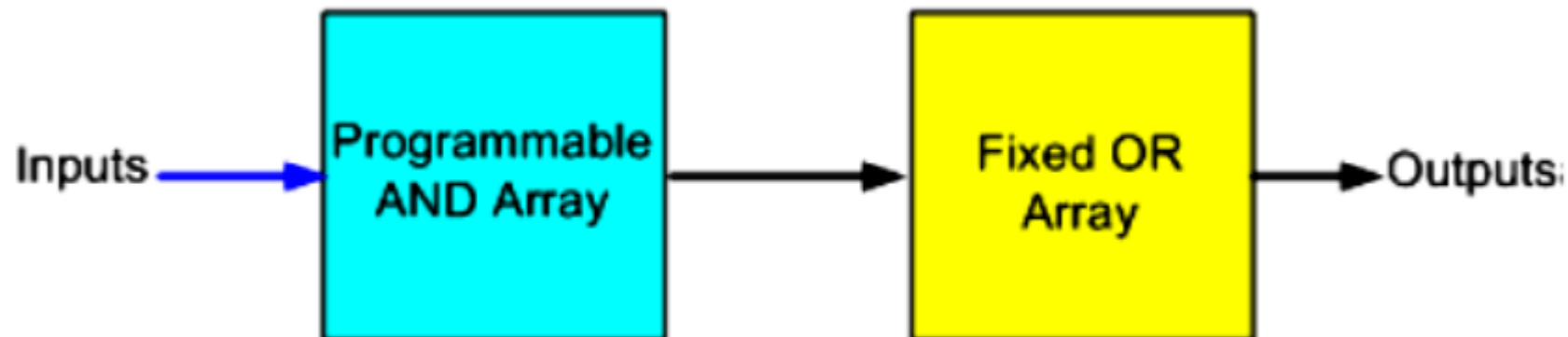
$$A(X,Y,Z) = \sum m(5,6,7)$$

$$B(X,Y,Z) = \sum m(3,5,6,7)$$

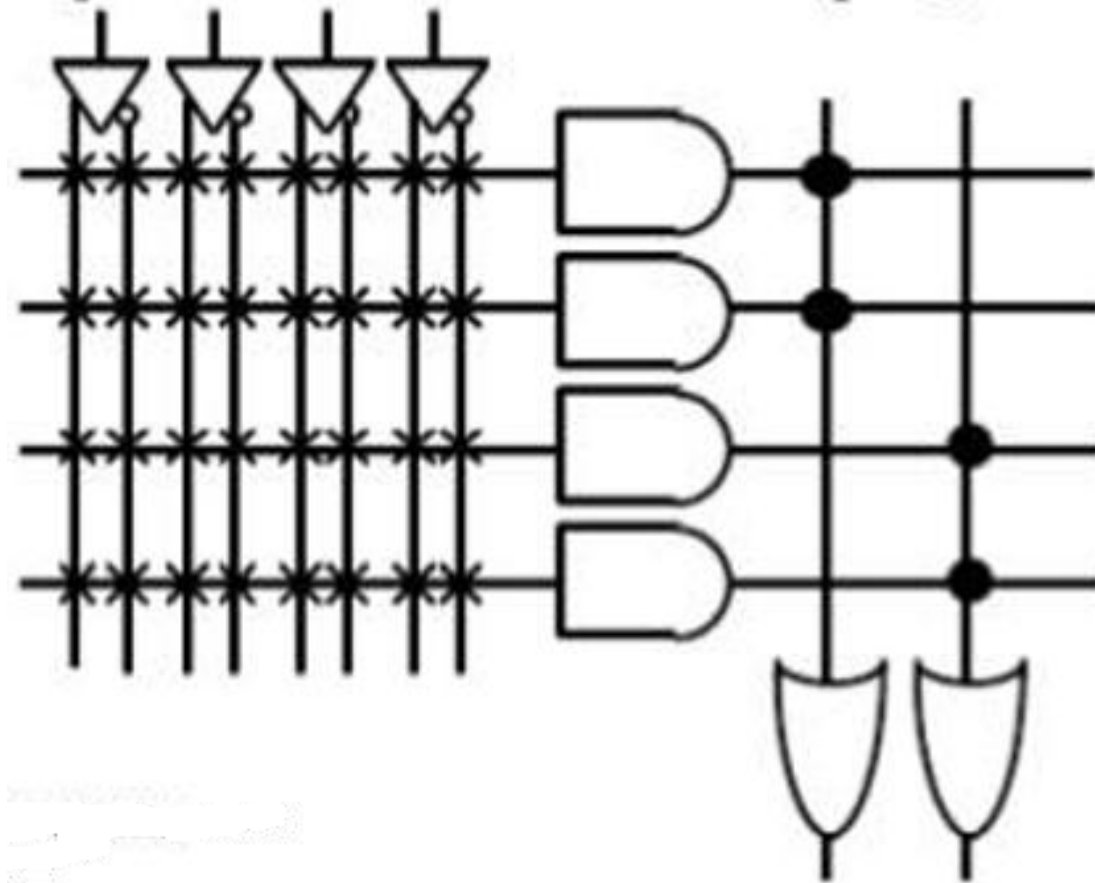


Programmable Array Logic

- The Programmable Array Logic (PAL) is also a type of PLD used to design and implement a variety of custom logic functions.
- These programmable array logic devices allow digital designers to develop complex logic structures with high flexibility and efficiency.
- Construction-wise, a PAL device consists of an array of programmable AND gates connected to a fixed array of OR gates.
- This array structure helps to implement various logic functions by interconnecting the input lines, AND gates and OR gates.



Programmable Array Logic



Numerical

- Implement this circuit using PAL.

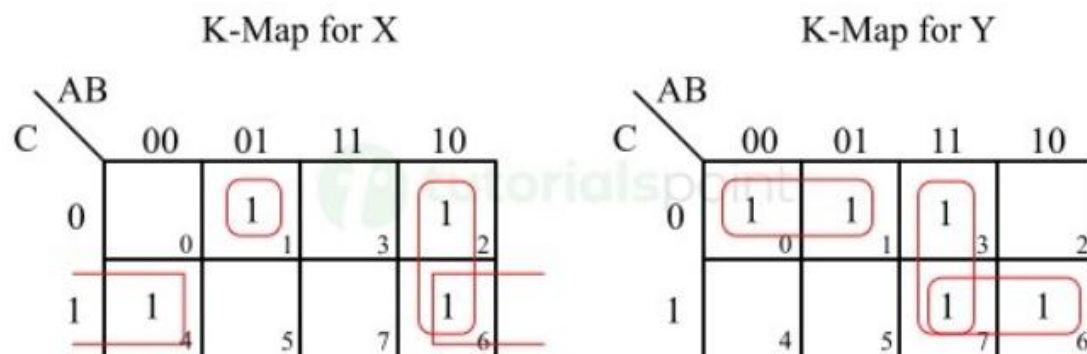
$$X(A,B,C)=\sum m(1,2,4,6)$$

$$Y(A,B,C)=\sum m(0,1,3,6,7)$$

Numerical

Solution

Obtaining the Boolean expressions for the given logic functions,



From these K-maps, we get,

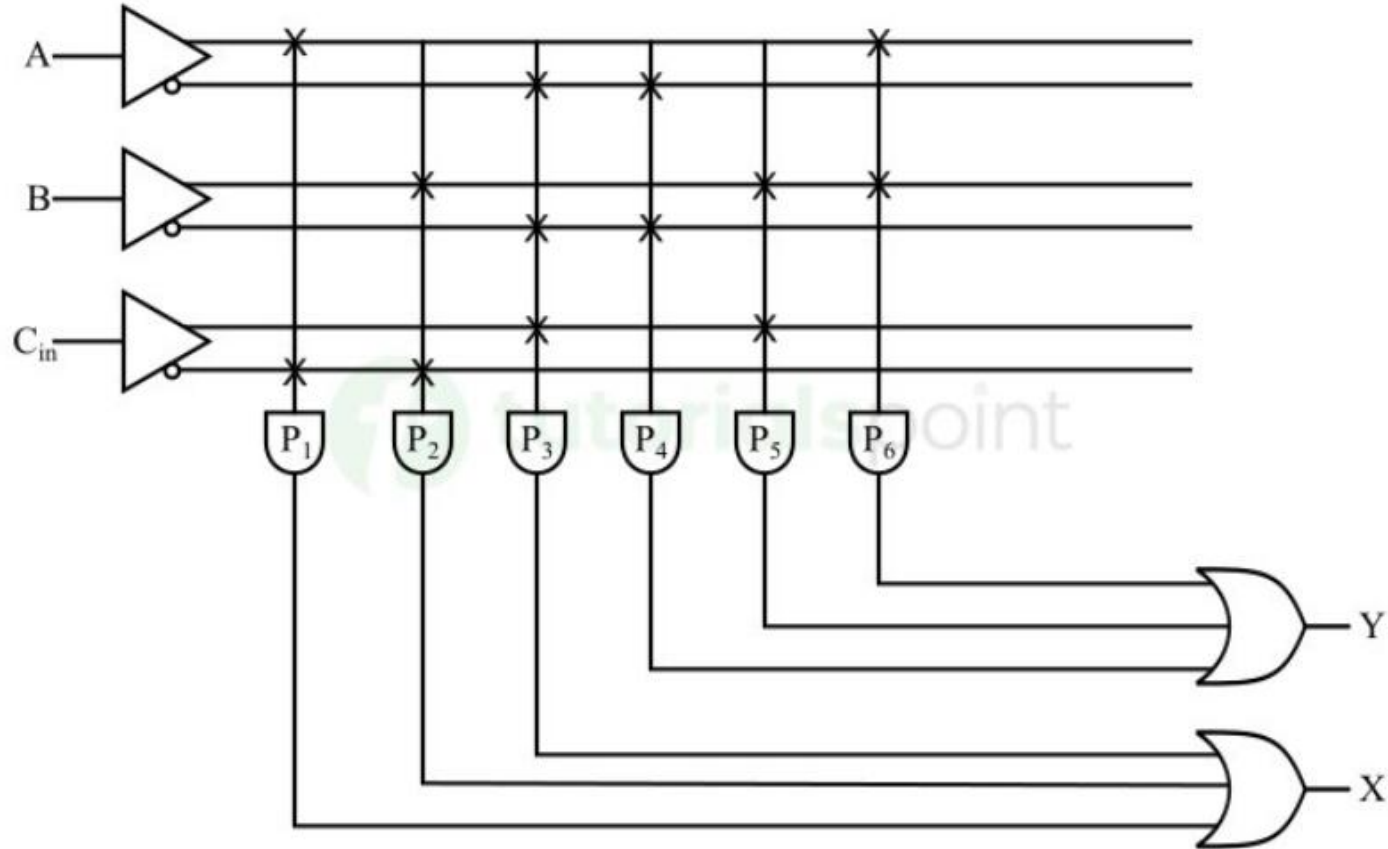
$$X = A\bar{C} + B\bar{C} + \bar{A}\bar{B}C$$

$$Y = \bar{A}\bar{B} + BC + AB$$

Numerical

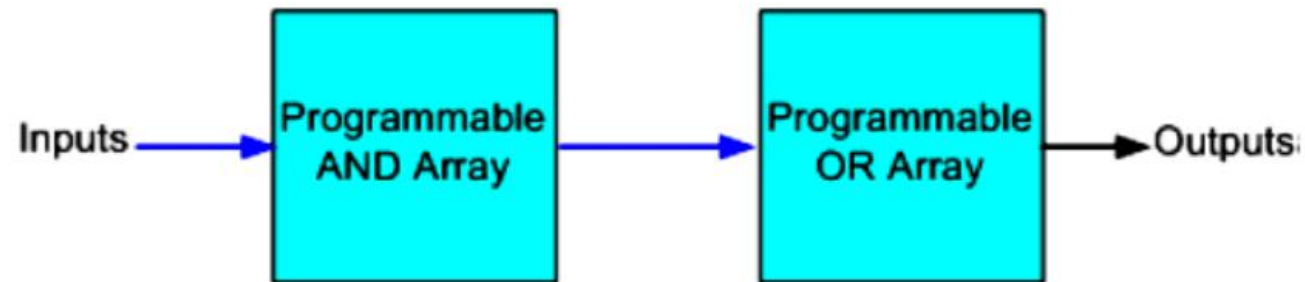
Product Terms		AND Gate Inputs		
		A	B	C
1	$A \bar{B}$	1	-	0
2	$B \bar{C}$	-	1	0
3	$\bar{A} \bar{B} C$	0	0	1
4	$\bar{A} \bar{B}$	0	0	-
5	$B C$	-	1	1
6	$A B$	1	1	-

Numerical



Programmable Logic Array

- In PLAs, instead of using a decoder as in PROMs, a number (k) of AND gates is used where $k < 2^n$, (n is the number of inputs).
- Each of the AND gates can be programmed to generate a product term of the input variables and does not generate all the minterms as in the ROM.
- The AND and OR gates inside the PLA are initially fabricated with the links (fuses) among them.
- The specific Boolean functions are implemented in sum of products form by opening appropriate links and leaving the desired connections.



(c) Programmable Logic Array (PLA) Device

Numerical

- Design a full-adder circuit using programmable logic array (PLA).

Solution:

Inputs			Outputs	
A	B	C _{in}	S	C _{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Numerical

From this truth table, the output sum (S) and the output carry (C_{out}) are given by,

$$S = \sum m(1, 2, 4, 7)$$

$$C_{out} = \sum m(3, 5, 6, 7)$$

Thus, their Boolean expressions will be,

K-Map for Sum (S)

		AB			
		00	01	11	10
C_{in}	0	0	1	3	2
	1	1		1	
		4	5	7	6

K-Map for Carry Out (C_{out})

		AB			
		00	01	11	10
C_{in}	0	0	1	1	2
	1		1	1	1
		4	5	7	6

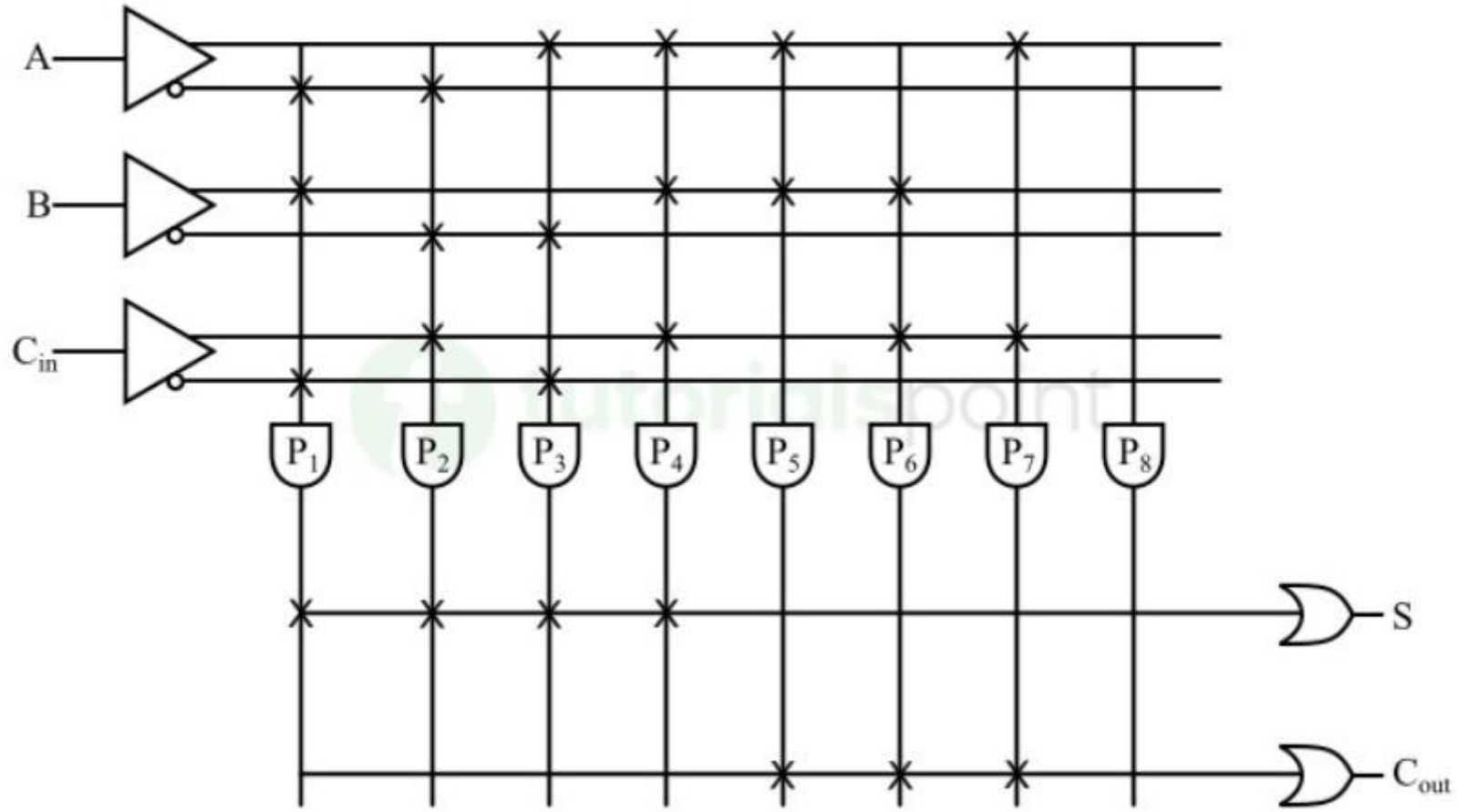
$$S = \overline{A} B \overline{C_{in}} + \overline{A} \overline{B} C_{in} + A \overline{B} \overline{C_{in}} + A B C_{in}$$

$$C_{out} = A B + B C_{in} + A C_{in}$$

Numerical

Sr.No	Product Terms	Inputs			Outputs	
		A	B	C _{in}	S	C _{out}
1	$\overline{A} B \overline{C_{in}}$	0	1	0	1	-
2	$\overline{A} \overline{B} C_{in}$	0	0	1	1	-
3	$A \overline{B} \overline{C_{in}}$	1	0	0	1	-
4	$A B C_{in}$	1	1	1	1	-
5	$A B$	1	1	-	-	1
6	$B C_{in}$	-	1	1	-	1
7	$A C_{in}$	1	-	1	-	1
					T	T

Numerical





SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya School of Engineering
(formerly K J Somaiya College of Engineering)



BEST WISHES FOR YOUR EXAMS!!!